

Module 12: Link State Routing and Dijkstra Algorithm

Unit 3: Network Layer

Global Topology & Dijkstra SPF

Link State Protocol Architecture (OSPF/IS-IS), 5 Execution Phases, LSP Reliable Flooding, Step-by-Step Dijkstra Numerical, and Master DVR vs LSR Comparison.

Link State Routing (LSR): LSP Reliable Flooding & Dijkstra's Algorithm

5 Phases of Link State Routing (OSPF / IS-IS)

1. Discover Neighbors:

Send HELLO packets to learn local interface neighbours.

2. Measure Link Metrics:

Send ECHO packets to measure round-trip delay, bandwidth.

3. Build Link State Packet (LSP):

Create LSP containing Node ID, sequence no, & neighbor costs.

4. Reliable Flooding of LSPs:

Distribute LSP to every router in the AS using ACKs.

5. Compute Shortest Path Tree:

Every node runs Dijkstra algorithm on its complete LSDB.

Structure of a Link State Packet (LSP):

- Source Router ID: R1
- Sequence Number: 0x0000004A (Prevents old packet reuse)
- Age: 3600 seconds (Decrement to prevent zombie entries)
- Neighbor 1: R2 (Metric = 2)
- Neighbor 2: R3 (Metric = 5)
- Neighbor 3: R4 (Metric = 1)

Dijkstra Shortest Path Tree (Source = Node U)

```
graph LR; U((U)) ---|2| V((V)); U ---|5| W((W)); V ---|1| X((X)); W ---|2| X
```

Step-by-Step Dijkstra Trace:

Step	Confirmed Set N'	D(V), p(V)	D(W), p(W)	D(X), p(X)
0	{U}	2, U*	5, U	inf
1	{U, V}	-	5, U	3, V* (2+1)
2	{U, V, X}	-	5, U*	-

Resulting Routing Table at Router U:

- Dest V: Cost = 2, Next Hop = V (Direct link)
- Dest X: Cost = 3, Next Hop = V | Dest W: Cost = 5, Next Hop = W

1. 5 Phases of Link State Routing

1. Discover neighbors via periodic HELLO packets.

2. Measure link cost/latency via ECHO test.

3. Construct Link State Packet (LSP) with Seq Number & Age.

4. Reliable Flooding of LSP to all routers in Autonomous System.

5. Execute Dijkstra's Algorithm independently on Link State Database (LSDB).

2. DVR vs LSR Master Comparison

Feature	Distance Vector (RIP)	Link State (OSPF)
Graph Knowledge	Local view only (Neighbor vectors)	Complete global topology (LSDB)
Update Sharing	Shares entire table with neighbors	Shares neighbor state with entire network
Convergence	Slow (Count-to-Infinity loops)	Fast & mathematically loop-free
Resource Overhead	Minimal memory & CPU	High CPU for Dijkstra graph execution